



OPERAÇÕES · GIT & CI/CD · INTERNO · EQUIPE DE DESENVOLVIMENTO

Guia de Operação Git — Ambiente Remoto

Procedimentos para manutenção do código, CI/CD e releases
aerosuite-br/aerosuite · Organização GitHub

Versão 1.0 · 18 de junho de 2026

Repositório: <https://github.com/aerosuite-br/aerosuite>

Sumário executivo

Este guia formaliza os procedimentos de interação com o ambiente Git remoto do projeto Aero Suite, após a criação da organização **aerosuite-br** e do repositório privado **aerosuite**. O documento cobre autenticação, fluxo de branches, integração contínua, secrets, releases e resolução de incidentes comuns.

master

PRODUÇÃO

desenv

DESENVOLVIMENTO

CIVERDE (BACKEND,
FRONTEND, MENU I18N)**2**

AMBIENTES GH

Checklist rápido — novo membro

- Conta GitHub na organização aerosuite-br
- gh auth login com escopo workflow
- Repositório clonado; branch desenv ativa
- git pull antes de cada sessão de trabalho
- Commits sem secrets nem artefatos de build
- Push para desenv; PR para master
- CI verde antes de merge
- Tag semântica após release em master

Visão geral e acessos

O código oficial do Aero Suite está centralizado na organização GitHub aerosuite-br. Este capítulo define quem acessa o quê e quais ferramentas instalar no posto de trabalho.

Recursos principais

Organização	@aerosuite-br
Repositório	aerosuite (privado)
URL	https://github.com/aerosuite-br/aerosuite
Branch produção	master
Branch desenvolvimento	desenv
CI	GitHub Actions — workflow CI
Ambientes	staging (desenv) · production (master)

Pré-requisitos no computador

- Git for Windows (ou Git no Linux/macOS) — <https://git-scm.com>
- GitHub CLI (gh) — <https://cli.github.com> — recomendado para autenticação e PRs
- Node.js 20+ e Java 21 — alinhados ao CI
- Conta GitHub convidada na organização aerosuite-br (aceitar convite por e-mail)

Nunca envie senhas, tokens PAT ou códigos de dispositivo por chat ou e-mail. A autenticação é feita apenas no seu terminal com `gh auth login`.

Autenticação GitHub CLI

Antes de push, pull ou criação de PR, o gh deve estar autenticado com escopos repo e workflow (obrigatório para enviar arquivos .github/workflows/).

Primeiro login

- 1 Abra PowerShell e execute: `gh auth login`
- 2 Escolha: GitHub.com → HTTPS → Login with a web browser
- 3 Copie o código exibido e autorize em <https://github.com/login/device>
- 4 Na tela "Authorize GitHub CLI", confirme acesso à organização aerosuite-br
- 5 Se o push falhar por workflow, execute: `gh auth refresh -h github.com -s workflow,repo,read:org,gist`
- 6 Valide: `gh auth status` (deve listar workflow entre os escopos)
- 7 Configure Git para usar credenciais do gh: `gh auth setup-git`

Verificação rápida

```
gh auth status
git remote -v
git branch -a
```

Clonar e configurar o repositório local

Desenvolvedores que ainda não possuem cópia local devem clonar a branch desenv e configurar identidade Git.

Clone inicial (HTTPS)

```
cd D:\Desenvolvimento
git clone https://github.com/aerosuite-br/aerosuite.git
cd aerosuite
git checkout desenv
git pull origin desenv
```

Remote já existente (atualizar URL)

```
git remote set-url origin https://github.com/aerosuite-br/aerosuite.git
git fetch origin
git branch -u origin/desenv desenv
```

Configuração de identidade (uma vez por máquina)

- `git config user.name "Seu Nome"`
- `git config user.email "seu.email@aerosuite.com.br"`
- Use o e-mail corporativo associado à conta GitHub

Estratégia de branches e fluxo diário

O trabalho cotidiano ocorre em desenv. Releases e deploy em produção partem de master, sempre via Pull Request após CI verde.

desenv —(PR + CI verde)→ **master** —(tag v*)→ **deploy produção**

Papéis das branches

desenv	Integração contínua, homologação UX, correções do dia a dia
master	Linha de produção — somente merge revisado
saas/**	Features SaaS isoladas (CI monitora)
tags v*	Marcos de release (ex.: v1.0.0)

Rotina diária (desenv)

- 1 git checkout desenv
- 2 git pull origin desenv
- 3 Implementar alterações e testar localmente
- 4 git status — revisar arquivos (não commitar target/, node_modules/, .env)
- 5 git add <arquivos relevantes>
- 6 git commit -m "tipo(escopo): descrição clara"
- 7 git push origin desenv
- 8 Aguardar CI verde em GitHub → Actions

No plano GitHub Free com repositório privado, branch protection automática exige upgrade. Até lá: disciplina de PR + CODEOWNERS + revisão humana em master.

Pull Requests e revisão

Toda entrada em master deve passar por Pull Request. O repositório inclui template de PR e CODEOWNERS para orientar revisão.

Abrir PR desenv → master

- 1 Com desenv atualizado e push feito, acesse o repositório no GitHub
- 2 Clique em "Compare & pull request" ou: `gh pr create --base master --head desenv`
- 3 Preencha o template: resumo, checklist CI, tipo de mudança
- 4 Aguarde jobs: Backend (Maven), Frontend (npm), Menu i18n audit
- 5 Após aprovação, use "Squash and merge" ou "Merge commit" conforme política da equipe
- 6 Delete branch temporária se aplicável

Boas práticas de commit

- Prefixos: fix, feat, chore, docs, test, refactor
- Mensagem no imperativo: "corrigir validação PN" não "corrigido..."
- Um commit = uma intenção lógica; evite commits gigantes
- Não incluir artefatos de build (backend/target, .tmp-*)
- UI nova: chaves i18n em pt-BR, en-US, es-ES, fr-FR

CI/CD e qualidade

O pipeline CI dispara em push e PR para master, desenv, develop e saas/**. E2E Playwright roda em PR manual/dispatch.

Workflows

ci.yml	Push/PR — Maven verify, npm build, testes unitários, a11y smoke
integration-smoke.yml	Manual — smoke HTTP contra API
covered-suite.yml	Manual — stack Docker + suite coberta

Jobs obrigatórios (CI padrão)

Backend (Maven)	mvn verify — testes + JaCoCo mínimo 18%
Frontend (npm)	npm ci, build, Vitest, a11y axe e fluxos F1–F12
Menu i18n audit	Cobertura menu.func.* / menu.section.*

Validar localmente antes do push

```
cd backend && mvn -B verify
cd frontend && npm ci && npm run build && npm run test:unit
powershell -File scripts/test/audit-menu-i18n.ps1 -SkipApi
```


Secrets, variáveis e ambientes

Secrets de teste e URLs de homologação estão em Settings → Secrets and variables → Actions. Ambientes staging e production restringem deploy por branch.

Secrets configurados (repo)

AEROSUITE_TEST_EMAIL	Login plataforma (tenant default)
AEROSUITE_TEST_PASSWORD	Senha de teste — não expor
AEROSUITE_TEST_TENANT	default
AEROSUITE_DEMO_* / MULTI_TENANT_*	Cenários demo e isolamento

Variables

AEROSUITE_WEB_URL	https://app.aerosuite.com.br
AEROSUITE_API_URL	https://app.aerosuite.com.br/api

Environments

staging	Deploy permitido a partir da branch desenv
production	Deploy permitido a partir da branch master

Nunca commitar .env, credenciais SMTP, chaves Stripe ou JWT. Use .env.example como referência.

Release e deploy produção

Após merge em master, marque a release com tag semântica e siga o checklist de deploy no servidor Hetzner.

Release versionada

- 1 `git checkout master && git pull origin master`
- 2 `git tag -a v1.0.0 -m "Release 1.0.0 — descrição"`
- 3 `git push origin v1.0.0`
- 4 Opcional: criar GitHub Release com notas a partir da tag

Deploy servidor (/opt/aerosuite)

- 1 SSH no VPS Ubuntu 24.04 (Hetzner)
- 2 `git clone` ou `git pull` em /opt/aerosuite (branch master ou tag)
- 3 Configurar `.env.production` (ver docs/DEPLOY-PRODUCAO.md)
- 4 `docker compose -f docker-compose.yml -f docker-compose.production.yml up -d --build`
- 5 Executar smoke pós-deploy (`scripts/deploy/pre-deploy-check.ps1`)

Solução de problemas

Erros frequentes e resolução objetiva.

Push rejeitado: OAuth App cannot update workflow without workflow scope

Execute `gh auth refresh -h github.com -s workflow,repo,read:org,gist` e autorize no navegador.

setup-node: unable to cache dependencies

Garanta que `frontend/package-lock.json` está commitado. Use `npm ci` no frontend.

CI falha em Menu i18n audit

Adicione chaves `menu.func.*` faltantes em `menu-i18n.ts` nas 4 línguas. Rode `audit-menu-i18n.ps1 -SkipApi`.

Conflito ao fazer pull

`git stash` → `git pull` → `git stash pop`. Resolva conflitos, teste, `commit` e `push`.

Remote incorreto (404)

`git remote set-url origin https://github.com/aerosuite-br/aerosuite.git`

Referências e contatos

Documentação complementar no repositório.

Documentos no repo

- docs/GITHUB-PRODUCAO-SETUP.md — bootstrap e checklist
- docs/CI-SECRETS.md — secrets Actions
- docs/DEPLOY-PRODUCAO.md — deploy Hetzner
- docs/COMANDOS_RAPIDOS_GITHUB.md — cola de comandos
- .github/pull_request_template.md — template de PR

Suporte

Diretor de TI	Wellem Lyra — wellemlyra@aerosuite.com.br
Organização GitHub	https://github.com/aerosuite-br
Actions / CI	https://github.com/aerosuite-br/aerosuite/actions